

PPU programmer reference

From NESdev Wiki

Contents

- 1 PPU Registers
- 2 Summary
- 3 MMIO registers
 - 3.1 PPUCTRL - Miscellaneous settings (\$2000 write)
 - 3.1.1 Vblank NMI
 - 3.1.2 Scrolling
 - 3.1.3 Master/slave mode and the EXT pins
 - 3.1.4 Bit 0 race condition
 - 3.2 PPUMASK - Rendering settings (\$2001 write)
 - 3.2.1 Rendering control
 - 3.2.2 Color control
 - 3.3 PPUSTATUS - Rendering events (\$2002 read)
 - 3.3.1 Vblank flag
 - 3.3.2 Sprite 0 hit flag
 - 3.3.3 Sprite overflow flag
 - 3.3.4 2C05 identifier
 - 3.4 OAMADDR - Sprite RAM address (\$2003 write)
 - 3.4.1 Values during rendering
 - 3.4.2 OAMADDR precautions
 - 3.5 OAMDATA - Sprite RAM data (\$2004 read/write)
 - 3.6 PPUSCROLL - X and Y scroll (\$2005 write)
 - 3.7 PPUADDR - VRAM address (\$2006 write)
 - 3.7.1 Note
 - 3.7.2 Palette corruption
 - 3.7.3 Bus conflict
 - 3.8 PPUDATA - VRAM data (\$2007 read/write)
 - 3.8.1 The PPUDATA read buffer
 - 3.8.2 Reading palette RAM
 - 3.8.3 Read conflict with DPCM samples
 - 3.9 OAMDMA - Sprite DMA (\$4014 write)
- 4 Internal registers
- 5 References
- 6 Pattern tables
- 7 Addressing
- 8 Display convention
- 9 OAM
- 10 OAM (Sprite) Data
 - 10.1 Byte 0 - Y position
 - 10.2 Byte 1 - Tile/index
 - 10.3 Byte 2 - Attributes
 - 10.4 Byte 3 - X position
- 11 Details
 - 11.1 DMA
 - 11.2 Sprite 0 hits

- 11.3 Sprite overlapping
- 11.4 Internal operation
- 11.5 Dynamic RAM decay
- 12 See also
- 13 References
- 14 Nametables
- 15 Mirroring
- 16 Background evaluation
- 17 See also
- 18 Attribute tables
- 19 Worked example
- 20 Glitches
- 21 Expansion
- 22 Palettes
- 23 Palette RAM
- 24 Color Value Significance (Hue / Value)
- 25 Palettes
 - 25.1 2C02
 - 25.2 2C07
 - 25.3 2C03 and 2C05
 - 25.4 2C05-99
 - 25.5 2C04
 - 25.5.1 RP2C04-0001
 - 25.5.2 RP2C04-0002
 - 25.5.3 RP2C04-0003
 - 25.5.4 RP2C04-0004
 - 25.5.5 Games compatible with multiple different PPUs
 - 25.5.6 LUT approach
- 26 Backdrop color
 - 26.1 Backdrop override
- 27 Color names
 - 27.1 Luma
 - 27.2 Chroma
 - 27.3 RGBI
- 28 See also
- 29 References
- 30 Memory map
 - 30.1 PPU memory map
- 31 Hardware mapping
- 32 OAM

PPU Registers

The PPU exposes eight memory-mapped registers to the CPU. These nominally sit at \$2000 through \$2007 in the CPU's address space, but because their addresses are incompletely decoded, they're mirrored in every 8 bytes from \$2008 through \$3FFF. For example, a write to \$3456 is the same as a write to \$2006.

The PPU starts rendering immediately after power-on or reset, but ignores writes to most registers (specifically \$2000, \$2001, \$2005 and \$2006) until reaching the pre-render scanline of the next frame; more specifically, for around 29658 NTSC CPU cycles or 33132 PAL CPU cycles, assuming the CPU and PPU are reset at the same time. See PPU power up state and Init code for details.

Summary

Common Name	Address	Bits	Type	Notes
PPUCTRL	\$2000	VPHB SINN	W	NMI enable (V), PPU master/slave (P), sprite height (H), background tile select (B), sprite tile select (S), increment mode (I), nametable select / X and Y scroll bit 8 (NN)
PPUMASK	\$2001	BGRs bMmG	W	color emphasis (BGR), sprite enable (s), background enable (b), sprite left column enable (M), background left column enable (m), greyscale (G)
PPUSTATUS	\$2002	VSO- ----	R	vblank (V), sprite 0 hit (S), sprite overflow (O); read resets write pair for \$2005/\$2006
OAMADDR	\$2003	AAAA AAAA	W	OAM read/write address
OAMDATA	\$2004	DDDD DDDD	RW	OAM data read/write
PPUSCROLL	\$2005	XXXX XXXX YYYY YYYY	Wx2	X and Y scroll bits 7-0 (two writes: X scroll, then Y scroll)
PPUADDR	\$2006	. .AA AAAA AAAA AAAA	Wx2	VRAM address (two writes: most significant byte, then least significant byte)
PPUDATA	\$2007	DDDD DDDD	RW	VRAM data read/write
OAMDMA	\$4014	AAAA AAAA	W	OAM DMA high address

Register types:

- **R** - Readable
- **W** - Writeable
- **x2** - Internal 2-byte state accessed by two 1-byte accesses

MMIO registers

The PPU has an internal data bus that it uses for communication with the CPU. This bus, called `_io_db` in Visual 2C02 and `PPUGenLatch` in FCEUX,^[1] behaves as an 8-bit dynamic latch due to capacitance of very long traces that run to various parts of the PPU. Writing any value to any PPU port, even to the nominally read-only `PPUSTATUS`, will fill this latch. Reading any readable port (`PPUSTATUS`, `OAMDATA`, or `PPUDATA`) also fills the latch with the bits read. Reading a nominally "write-only" register returns the latch's current value, as do the unused bits of `PPUSTATUS`. At least one bit in this value decays after 3ms to 30ms, faster when the PPU is warm.^[2]

PPUCTRL - Miscellaneous settings (\$2000 write)

```
7 bit 0
-----
VPHB SINN
|||| |||
|||| |++- Base nametable address
|||| | (0 = $2000; 1 = $2400; 2 = $2800; 3 = $2C00)
|||| |++- VRAM address increment per CPU read/write of PPUDATA
|||| | (0: add 1, going across; 1: add 32, going down)
|||| |++- Sprite pattern table address for 8x8 sprites
|||| | (0: $0000; 1: $1000; ignored in 8x16 mode)
```

```

|||+----- Background pattern table address (0: $0000; 1: $1000)
||+----- Sprite size (0: 8x8 pixels; 1: 8x16 pixels - see PPU OAM#Byte 1)
|+----- PPU master/slave select
|         (0: read backdrop from EXT pins; 1: output color on EXT pins)
+----- Vblank NMI enable (0: off, 1: on)

```

PPUCTRL (the "control" or "controller" register) contains a mix of settings related to rendering, scroll position, vblank NMI, and dual-PPU configurations. After power/reset, writes to this register are ignored until the first pre-render scanline.

Vblank NMI

Enabling NMI in PPUCTRL causes the NMI handler to be called at the start of vblank (scanline 241, dot 1). This provides a reliable time source for software so it can run at the display's frame rate, and it signals vblank to the software. Vblank is the only time with rendering enabled that the software can send data to VRAM and OAM, and this NMI is the *only* reliable way to detect vblank; polling the vblank flag in PPUSTATUS can miss vblank entirely.

Changing NMI enable from 0 to 1 while the vblank flag in PPUSTATUS is 1 will immediately trigger an NMI. This happens during vblank if the PPUSTATUS register has not yet been read. It can result in graphical glitches by making the NMI routine execute too late in vblank to finish on time, or cause the game to handle more frames than have actually occurred. To avoid this problem, it is prudent to read PPUSTATUS first to clear the vblank flag before enabling NMI in PPUCTRL.

Scrolling

The current nametable bits in PPUCTRL bits 0 and 1 can equivalently be considered the most significant bit of the scroll coordinates, which are 9 bits wide (see Nametables and PPUSCROLL):

```

7 bit 0
----
.... ..YX
    ||
    |+- X scroll position bit 8 (i.e. add 256 to X)
    +-- Y scroll position bit 8 (i.e. add 240 to Y)

```

These two bits go to the same internal t register as the values written to PPUSCROLL, and they must be written alongside PPUSCROLL in order to fully specify the scroll position.

Master/slave mode and the EXT pins

Bit 6 of PPUCTRL should never be set on stock consoles because it may damage the PPU.

When this bit is clear (the usual case), the PPU gets the palette index for the backdrop color from the EXT pins. The stock NES grounds these pins, making palette index 0 the backdrop color as expected. A secondary picture generator connected to the EXT pins would be able to replace the backdrop with a different image using colors from the background palette, which could be used for features such as parallax scrolling.

Setting bit 6 causes the PPU to output the lower four bits of the palette memory index on the EXT pins for each pixel. Since only four bits are output, background and sprite pixels can't normally be distinguished this way. Setting this bit does not affect the image in the PPU's composite video output. As the EXT pins are grounded on an unmodified NES, setting bit 6 is discouraged as it could potentially damage the chip whenever it outputs a non-zero

pixel value (due to it effectively shorting Vcc and GND together). Note that EXT output for transparent pixels is not a backdrop color as normal, but rather entry 0 of that background sliver's palette. When rendering is disabled, EXT output is always index 0 regardless of backdrop override.

Bit 0 race condition

Be careful when writing to this register outside vblank if using a horizontal nametable arrangement (a.k.a. vertical mirroring) or 4-screen VRAM. For specific CPU-PPU alignments, a write that starts (<https://forums.nesdev.org/viewtopic.php?p=112424#p112424>) on dot 257 will cause only the next scanline to be erroneously drawn from the left nametable. This can cause a visible glitch, and it can also interfere with sprite 0 hit for that scanline (by being drawn with the wrong background).

The glitch has no effect in horizontal or one-screen mirroring because the left and right nametables are identical. Only writes that start on dot 257 and continue through dot 258 can cause this glitch: any other horizontal timing is safe. The glitch specifically writes the value of open bus to the register, which will almost always be the upper byte of the address. Writing to this register or the mirror of this register at \$2100 according to the desired nametable appears to be a functional workaround (<https://forums.nesdev.org/viewtopic.php?p=230434#p230434>).

This produces an occasionally visible glitch in *Super Mario Bros.* when the program writes to PPUCTRL at the end of game logic. It appears to be turning NMI off during game logic and then turning NMI back on once the game logic has finished in order to prevent the NMI handler from being called again before the game logic finishes. Another workaround is to use a software flag to prevent NMI reentry, instead of using the PPU's NMI enable.

PPUMASK - Rendering settings (\$2001 write)

```

7 bit 0
-----
BGRs bMmG
|||| |||
|||| |||+- Greyscale (0: normal color, 1: greyscale)
|||| |||+- 1: Show background in leftmost 8 pixels of screen, 0: Hide
|||| |||+- 1: Show sprites in leftmost 8 pixels of screen, 0: Hide
|||| +-+ 1: Enable background rendering
|||| +-+ 1: Enable sprite rendering
||+-+----- Emphasize red (green on PAL/Dendy)
|+-+----- Emphasize green (red on PAL/Dendy)
+-+----- Emphasize blue

```

PPUMASK (the "mask" register) controls the rendering of sprites and backgrounds, as well as color effects. After power/reset, writes to this register are ignored until the first pre-render scanline.

Most commonly, PPUMASK is set to \$00 outside of gameplay to allow transferring a large amount of data to VRAM, and \$1E during gameplay to enable all rendering with no color effects.

Rendering control

Rendering is the PPU's process of actively fetching memory and drawing an image to the screen. Rendering as a whole is enabled as long as one or both of sprite and background rendering is enabled in PPUMASK. If one component is enabled and the other is not, the disabled component is simply treated as transparent; the rendering process is otherwise unaffected. When both components are disabled via bits 3 and 4, the rendering process stops and the PPU displays the backdrop color.

During rendering, the PPU is actively using VRAM and OAM. This prevents the CPU from being able to access VRAM via PPUDATA or OAM via OAMDATA, so these accesses must be done outside of rendering: either during vblank (for data transfers during gameplay) or with rendering turned off (for large data transfers, such as when loading a level). To avoid numerous hardware bugs and limitations, it is generally recommended that rendering be turned on or off only during vblank. This can be done by writing the desired PPUMASK value to a variable rather than the register itself and then only copying that variable to PPUMASK during vblank in the NMI handler.

The PPU can optionally hide sprites and backgrounds in just the leftmost 8 pixels of the screen, making them transparent and thus drawing the backdrop color there. For sprites, this can be useful to avoid sprite pop-in, a limitation where sprites cannot partially hang off the left edge of the screen like they can off the right edge. For backgrounds, this can eliminate tile artifacts and reduce attribute artifacts when scrolling horizontally with either a vertical or one-screen nametable arrangement, as these arrangements do not allow hiding the scroll seam off-screen. Note that the backdrop color may not match the color used by the art for the background, so disabling the left column may be more distracting than minor artifacts.

Notes:

- Writing to PPUDATA during rendering can corrupt VRAM, so writes must be done in vblank or with rendering disabled in PPUMASK bits 3 and 4.
- Sprite 0 hit does not trigger in any area where the background or sprites are disabled.
- Toggling rendering takes effect approximately 3-4 dots after the write. This delay is required by Battletoads to avoid a crash.
- Toggling rendering mid-screen often corrupts 1 row of OAM and draws incorrect sprites for the current and next scanline. (See: Errata)
- Turning rendering off mid-screen can corrupt palette RAM if the low 14 bits of the internal v register have a value between \$3C00-\$3FFF.
- Turning rendering on late causes the dot at the end of pre-render to never be skipped, which can cause dot crawl on stationary screens.
- Turning rendering on late causes the PPU to have an incorrect scroll value unless it is set manually with a complicated series of writes.

Color control

Greyscale mode forces all colors to be a shade of grey or white. This is done by bitwise ANDing the color with \$30, causing all colors to come from the grey column (\$00, \$10, \$20, \$30), which notably lacks a black color. Note that this AND behavior means that RGB PPUs with scrambled colors (the 2C04 series) do not actually get shades of grey, but rather whatever colors are in the \$x0 column. When reading from palette RAM, the returned value reflects this AND behavior, but the underlying data is preserved. Palette writes function normally regardless of greyscale mode.

Color emphasis causes a color tint effect that works by darkening the other two color components, making the selected component comparatively brighter and thus emphasized. Emphasizing all 3 components simply dims all colors. This works independently of greyscale, allowing greys to be tinted. Note that PAL and Dendy PPUs have a different emphasis bit order, so ports and dual-region games should reorder the bits. Furthermore, emphasis on RGB PPUs is completely different, instead maximizing the brightness of the emphasized component and producing a completely white screen when all components are emphasized. RGB emphasis is far less useful and generally best avoided.

PPUSTATUS - Rendering events (\$2002 read)

```
7 bit 0
-----
VS0x xxxx
```

```

|||| ||||
|||+----- (PPU open bus or 2C05 PPU identifier)
||+----- Sprite overflow flag
|+----- Sprite 0 hit flag
+----- Vblank flag, cleared on read. Unreliable; see below.

```

PPUSTATUS (the "status" register) reflects the state of rendering-related events and is primarily used for timing. The three flags in this register are automatically cleared on dot 1 of the prerender scanline; see PPU rendering for more information on the set and clear timing.

Reading this register has the side effect of clearing the PPU's internal w register. It is commonly read before writes to PPUSCROLL and PPUADDR to ensure the writes occur in the correct order.

Vblank flag

The vblank flag is set at the start of vblank (scanline 241, dot 1). Reading PPUSTATUS will return the current state of this flag and then clear it. If the vblank flag is not cleared by reading, it will be cleared automatically on dot 1 of the prerender scanline.

Reading the vblank flag is not a reliable way to detect vblank. NMI should be used, instead. Reading the flag on the dot before it is set (scanline 241, dot 0) causes it to read as 0 and be cleared, so polling PPUSTATUS for the vblank flag can miss vblank and cause games to stutter. NMI is also suppressed when this occurs, and may even be suppressed by reads landing on the following dot or two. On NTSC and PAL, it is guaranteed that the flag cannot be dropped two frames in a row, but on Dendy, it is possible for it to happen every frame, crashing the game. Using NMI ensures that software correctly detects vblank every frame. It is also required by PlayChoice-10, which will reject the game if NMI is disabled for too long. Polling the vblank flag is still required while booting up the console, but timing at this point is not critical (see Init code for more information on booting safely).

The vblank flag is used in the generation of NMI, and enabling NMI while this flag is 1 will cause an immediate NMI (see PPUCTRL).

Sprite 0 hit flag

Sprite 0 hit is a hardware collision detection feature that detects pixel-perfect collision between the first sprite in OAM (sprite 0) and the background. The sprite 0 hit flag is immediately set when any opaque pixel of sprite 0 overlaps any opaque pixel of background, regardless of sprite priority. 'Opaque' means that the pixel is not 'transparent' — that is, its two pattern bits are not %00. The flag stays set until dot 1 of the prerender scanline; thus, it can only detect one collision per frame.

Although this flag detects collision, it is primarily used for timing. Many games place sprite 0 at a fixed location on the screen and poll this flag until it becomes set. This allows the CPU to know its approximate location on the screen so it can time mid-screen writes to hardware registers. Commonly, this is used to change the scroll position mid-screen to allow for a background-based HUD, like in *Super Mario Bros*. However, some modern homebrew games use this for actual collision, such as *Lunar Limit* (<https://forums.nesdev.org/viewtopic.php?t=15850>) and *Irritating Ship* (<https://fiskbit.itch.io/irritating-ship>).

Sprite 0 hit cannot detect collision at X=255, nor anywhere where either sprites or backgrounds are disabled via PPUMASK. This includes X=0.7 when the leftmost 8 pixels are hidden. However, it is not affected by the cropping on the left and right edges on PAL.

There are some important considerations when using this flag for timing:

- Because sprite 0 hit is not cleared until the prerender scanline, software can potentially mistake the previous frame's hit as being from the current frame. Therefore, it may be necessary to poll the flag until it becomes clear before then polling for it to be set again.
- If a game expects sprite 0 hit to occur and it does not, this often results in a crash. If there is any risk that the hit may not occur (perhaps because an overlap may not happen when scrolling or because it relies on precise mid-screen timings that may vary across power cycles, consoles, or emulators), it can be critical to have another way to exit the poll loop. For example, this may be done by also polling the vblank flag or having the NMI handler check if the game is still polling for sprite 0 hit.
- Games often don't handle sprite 0 hit on lag frames, preventing the mid-screen event from occurring. A common result of this is HUD flickering during lag. Handling sprite 0 hit in the NMI handler, at least on lag frames, can work around this.

Sprite overflow flag

The sprite overflow flag was intended to be set any time there are more than 8 sprites on a scanline. Unfortunately, the logic for detecting this does not work correctly, resulting in the PPU checking incorrect indices in OAM when searching for a 9th sprite. This produces both false positives and false negatives. See PPU sprite evaluation for details on its incorrect behavior. In practice, sprite overflow is usually used for timing like sprite 0 hit, but because of its buggy behavior and its cost of 9 sprite tiles, it is generally only used when more than one timing source is required. Like sprite 0 hit, this flag is cleared at the start of the prerender scanline and can only be set once per frame.

Using sprite overflow is often a last resort. When mapper IRQs are not available, the DMC IRQ can be an effective alternative for timing, albeit complicated to use.

2C05 identifier

The 2C05 series of arcade PPUs returns an identifier in bits 4-0 instead of PPU open bus. This value is checked by games as a form of copy protection. Note that this does not apply to the consumer 2C05-99, which returns open bus as usual. While we haven't yet collected data directly from the PPUs, 2C05 games expect the following values:

PPU	Mask	Value
2C05-02	\$3F	\$3D
2C05-03	\$1F	\$1C
2C05-04	\$1F	\$1B

OAMADDR - Sprite RAM address (\$2003 write)

```

7 bit 0
-----
AAAA AAAA
|||| ||||
+++++----- OAM address

```

Write the address of OAM you want to access here. Most games just write \$00 here and then use OAMDMA. (DMA is implemented in the 2A03/7 chip and works by repeatedly writing to OAMDATA)

Values during rendering

OAMADDR is set to 0 during each of ticks 257–320 (the sprite tile loading interval) of the pre-render and visible scanlines. This also means that at the end of a normal complete rendered frame, OAMADDR will always have returned to 0.

If rendering is enabled mid-scanline^[3], there are further consequences of an OAMADDR that was not set to 0 before OAM sprite evaluation begins at tick 65 of the visible scanline. The value of OAMADDR at this tick determines the starting address for sprite evaluation for this scanline, which can cause the sprite at OAMADDR to be treated as it was sprite 0, both for sprite-0 hit and priority. If OAMADDR is unaligned and does not point to the Y position (first byte) of an OAM entry, then whatever it points to (tile index, attribute, or X coordinate) will be reinterpreted as a Y position, and the following bytes will be similarly reinterpreted. No more sprites will be found once the end of OAM is reached, effectively hiding any sprites before the starting OAMADDR.

OAMADDR precautions

On the 2C02G, writes to OAMADDR corrupt OAM. The exact corruption isn't fully described, but this usually seems to copy sprites 8 and 9 (address \$20) over the 8-byte row at the target address. The source address for this copy seems to come from the previous value on the CPU BUS (most often \$20 from the \$2003 operand).^{[3][4]} There may be other possible behaviors as well. This can then be worked around by writing all 256 bytes of OAM, though due to the limited time before OAM decay will begin this should normally be done through OAMDMA.

It is also the case that if OAMADDR is not less than eight when rendering starts, the eight bytes starting at OAMADDR & 0xF8 are copied to the first eight bytes of OAM; it seems likely that this is related. On the Dendy, the latter bug is required for 2C02 compatibility.

It is known that in the 2C03, 2C04, 2C05^[5], and 2C07, OAMADDR works as intended. It is not known whether this bug is present in all revisions of the 2C02.

OAMDATA - Sprite RAM data (\$2004 read/write)



Write OAM data here. Writes will increment OAMADDR after the write; reads do not. Reads during vertical or forced blanking return the value from OAM at that address.

Do not write directly to this register in most cases. Because changes to OAM should normally be made only during vblank, writing through OAMDATA is only effective for partial updates, as it is too slow to update all of OAM within one vblank interval, and as described above, partial writes cause corruption. Most games use the DMA feature through OAMDMA instead.

- Reading OAMDATA while the PPU is rendering will expose internal OAM accesses during sprite evaluation and loading; *Micro Machines* does this.
- Writes to OAMDATA during rendering (on the pre-render line and the visible lines 0–239, provided either sprite or background rendering is enabled) do not modify values in OAM, but do perform a glitchy increment of OAMADDR, bumping only the high 6 bits (i.e., it bumps the $[n]$ value in PPU sprite evaluation – it's plausible that it could bump the low bits instead depending on the current status of sprite evaluation). This

extends to DMA transfers via OAMDMA, since that uses writes to \$2004. For emulation purposes, it is probably best to completely ignore writes during rendering.

- It used to be thought that reading from this register wasn't reliable^[6], however more recent evidence seems to suggest that this is solely due to corruption by OAMADDR writes.
- In the oldest instantiations of the PPU, as found on earlier Famicoms and NESes, this register is not readable^[7]. The readability was added on the RP2C02G, found on most NESes and later Famicoms.^[8]
- In the 2C07, sprite evaluation can *never* be fully disabled, and will always start 24 scanlines after the start of vblank^[9] (same as when the prerender scanline would have been on the 2C02). As such, any updates to OAM should be done within the first 24 scanlines after the 2C07 signals vertical blanking.

PPUSCROLL - X and Y scroll (\$2005 write)

```
1st write
7 bit 0
----
XXXX XXXX
|||| ||||
+++++----- X scroll bits 7-0 (bit 8 in PPUCTRL bit 0)

2nd write
7 bit 0
----
YYYY YYYY
|||| ||||
+++++----- Y scroll bits 7-0 (bit 8 in PPUCTRL bit 1)
```

This register is used to change the scroll position, telling the PPU which pixel of the nametable selected through PPUCTRL should be at the top left corner of the rendered screen. PPUSCROLL takes two writes: the first is the X scroll and the second is the Y scroll. Whether this is the first or second write is tracked internally by the w register, which is shared with PPUADDR. Typically, this register is written to during vertical blanking to make the next frame start rendering from the desired location, but it can also be modified during rendering in order to split the screen. Changes made to the vertical scroll during rendering will only take effect on the next frame. Together with the nametable bits in PPUCTRL, the scroll can be thought of as 9 bits per component, and PPUCTRL must be updated along with PPUSCROLL to fully specify the scroll position.



The PPU scroll registers share internal state with the PPU address registers. Because of this, PPUSCROLL and the nametable bits in PPUCTRL should be written *after* any writes to PPUADDR.

After reading PPUSTATUS to clear w (the write latch), write the horizontal and vertical scroll offsets to PPUSCROLL just before turning on the screen:

```
; Set the high bit of X and Y scroll.
lda ppuctrl_value
ora current_nametable
sta PPUCTRL

; Set the low 8 bits of X and Y scroll.
bit PPUSTATUS
lda cam_position_x
sta PPUSCROLL
lda cam_position_y
sta PPUSCROLL
```

Horizontal offsets range from 0 to 255. "Normal" vertical offsets range from 0 to 239, while values of 240 to 255 cause the attributes data at the end of the current nametable to be used incorrectly as tile data. The PPU normally skips from 239 to 0 of the next nametable automatically, so these "invalid" scroll positions only occur if explicitly written.

By changing the scroll values here across several frames and writing tiles to newly revealed areas of the nametables, one can achieve the effect of a camera panning over a large background.

PPUADDR - VRAM address (\$2006 write)

```

1st write  2nd write
15 bit  8  7  bit  0
-----
..AA AAAA  AAAA AAAA
  || ||||  |||| ||||
+++++-----+ VRAM address

```

Because the CPU and the PPU are on separate buses, neither has direct access to the other's memory. The CPU writes to VRAM through a pair of registers on the PPU by first loading an address into PPUADDR and then writing data repeatedly to PPUDATA. The VRAM address only needs to be set once for every series of data writes because each PPUDATA access automatically increments the address by 1 or 32, as configured in PPUCTRL.

The 16-bit address is written to PPUADDR one byte at a time, high byte first. Whether this is the first or second write is tracked by the PPU's internal w register, which is shared with PPUSCROLL. If w is not 0 or its state is not known, it must be cleared by reading PPUSTATUS before writing the address. For example, to set the VRAM address to \$2108 after w is known to be 0:

```

lda #$21
sta PPUADDR
lda #$08
sta PPUADDR

```

The PPU address space is 14-bit, spanning \$0000–\$3FFF. Bits 14 and 15 of the value written to this register are ignored. However, bit 14 of the internal t register that holds the data written to PPUADDR is forced to 0 when writing the PPUADDR high byte. This detail doesn't matter when using PPUADDR to set a VRAM address, but is an important limitation when using it to control mid-screen scrolling (see PPU scrolling for more information).

Note

Access to PPUSCROLL and PPUADDR during screen refresh produces interesting raster effects; the starting position of each scanline can be set to any pixel position in nametable memory. For more information, see PPU scrolling.

Palette corruption

In specific circumstances, entries of the PPU's palette can be corrupted. It's unclear exactly how or why this happens, but all revisions of the NTSC PPU seem to be at least somewhat susceptible.^[10]

When done writing to palette memory, the workaround is to always

1. Update the address, if necessary, so that it's pointing at \$3F00, \$3F10, \$3F20, or any other mirror.
2. Only then change the address to point outside of palette memory.

A code fragment to implement this workaround is present in vast numbers of games:^[11]

```
lda #$3F
sta PPUADDR
lda #0
sta PPUADDR
sta PPUADDR
sta PPUADDR
```

Bus conflict

During raster effects, if the second write to PPUADDR happens at specific times, at most one axis of scrolling will be set to the bitwise AND of the written value and the current value. The only safe time to finish the second write is during blanking; see PPU scrolling for more specific timing. [1] (<https://forums.nesdev.org/viewtopic.php?p=230391#p230391>)

PPUDATA - VRAM data (\$2007 read/write)

```
7 bit 0
-----
DDDD DDDD
|||| ||||
+++++----- VRAM data
```

VRAM read/write data register. After access, the video memory address will increment by an amount determined by bit 2 of \$2000.

When the screen is turned off by disabling the background/sprite rendering flag with the PPUMASK or during vertical blank, data can be read from or written to VRAM through this port. Since accessing this register increments the VRAM address, it should not be accessed outside vertical or forced blanking because it will cause graphical glitches, and if writing, write to an unpredictable address in VRAM. However, a handful of games are known to read from PPUDATA during rendering, causing scroll position changes. See PPU scrolling and Tricky-to-emulate games.

VRAM reading and writing shares the same internal address register that rendering uses. Therefore, after loading data into video memory, the program should reload the scroll position afterwards with PPUSCROLL and PPUCTRL (bits 1-0) writes in order to avoid wrong scrolling.

The PPUDATA read buffer

Reading from PPUDATA does not directly return the value at the current VRAM address, but instead returns the contents of an internal read buffer. This read buffer is updated on every PPUDATA read, but only **after** the previous contents have been returned to the CPU, effectively delaying PPUDATA reads by one. This is because PPU bus reads are too slow and cannot complete in time to service the CPU read. Because of this read buffer, after the VRAM address has been set through PPUADDR, one should first read PPUDATA to prime the read buffer (ignoring the result) before then reading the desired data from it.

Note that the read buffer is updated **only** on PPUDATA reads. It is not affected by writes or other PPU processes such as rendering, and it maintains its value indefinitely until the next read.

Reading palette RAM

Later PPUs added an unreliable feature for reading palette data from \$3F00-\$3FFF. These reads work differently than standard VRAM reads, as palette RAM is a separate memory space internal to the PPU that is overlaid onto the PPU address space. The referenced 6-bit palette data is returned immediately instead of going to the internal read buffer, and hence no priming read is required. Simultaneously, the PPU also performs a normal read from PPU memory at the specified address, "underneath" the palette data, and the result of this read goes into the read buffer as normal. The old contents of the read buffer are discarded when reading palettes, but by changing the address to point outside palette RAM and performing one read, the contents of this shadowed memory (usually mirrored nametables) can be accessed. On PPUs that do not support reading palette RAM, this memory range behaves the same as the rest of PPU memory.

This feature is supported by the 2C02G, 2C02H, and PAL PPUs. The byte returned when reading palettes contains PPU open bus in the top 2 bits, and the value is returned after it is modified by greyscale mode, which clears the bottom 4 bits if enabled. Unfortunately, on some consoles, palette reads can be corrupted on one of the 4 CPU/PPU alignments relative to the master clock. This corruption depends on when the PPU /CS signal that indicates register access is deasserted, which varies by console. Combined with this feature not being present in all PPUs, developers should not rely on reading from palette RAM.

Read conflict with DPCM samples

If currently playing DPCM samples, there is a chance that an interruption from the APU's sample fetch will cause an extra read cycle if it happened at the same time as an instruction that reads \$2007. This will cause an extra increment and a byte to be skipped over, resulting in the wrong data being read. See: APU DMC

OAMDMA - Sprite DMA (\$4014 write)



OAMDMA is a CPU register that suspends the CPU so it can quickly copy a page of CPU memory to PPU OAM using DMA. It always copies 256 bytes and the source address always starts page-aligned (ending in \$00). The value written to this register is the high byte of the source address, and the copy begins on the cycle immediately after the write. The copy takes 513 or 514 cycles and is implemented as 256 pairs of a read from CPU memory and a write to OAMDATA. Because vblank is so short and because changing OAMADDR often corrupts OAM, OAM DMA is normally the only realistic option for updating sprites each frame. 0 should be written to OAMADDR before initiating DMA to ensure the data is properly aligned and to avoid corruption.^[4] While OAM DMA is possible to do mid-frame while rendering is disabled, it is normally only done in vblank.

OAM consists of dynamic RAM (DRAM) which decays if not refreshed often enough, and this requires different considerations on NTSC and PAL. Refresh happens automatically any time a row of DRAM is read or written, so it is refreshed every scanline during rendering by the sprite evaluation process. On NTSC, vblank is short enough that OAM will not decay before rendering begins again, so OAM DMA can be done anytime in vblank. On PAL, vblank is much longer, so to avoid decay during that time, the PPU automatically performs a forced refresh starting 24 scanlines after NMI, during which OAM cannot be written. This means that OAM DMA is limited to the start of vblank on PAL. Note that NTSC vblank is shorter than 24 PAL scanlines, so NTSC-compatible NMI handlers will finish before the forced refresh and therefore should work on PAL regardless of their OAM DMA timing. In either

case, OAM does not decay if it is not updated during vblank, and in fact it should generally not be updated on lag frames (frames where the CPU did not finish its work before vblank) to avoid copying incomplete sprite data to the PPU.

Internal registers

The PPU also has 4 internal registers, described in detail on PPU scrolling:

- **v**: During rendering, used for the scroll position. Outside of rendering, used as the current VRAM address.
- **t**: During rendering, specifies the starting coarse-x scroll for the next scanline and the starting y scroll for the screen. Outside of rendering, holds the scroll or VRAM address before transferring it to v.
- **x**: The fine-x position of the current scroll, used during rendering alongside v.
- **w**: Toggles on each write to either PPUSCROLL or PPUADDR, indicating whether this is the first or second write. Clears on reads of PPUSTATUS. Sometimes called the 'write latch' or 'write toggle'.

References

1. ppu.cpp (<http://sourceforge.net/p/fceultra/code/HEAD/tree/fceu/trunk/src/ppu.cpp#l183>) by Bero and Xodnizel
2. replies to lidnariq's PPU decay test ROM (<https://forums.nesdev.org/viewtopic.php?t=24639>)
3. OAMDATA \$2003 corruption clarification? (<https://forums.nesdev.org/viewtopic.php?p=285674#p285674>) - forum thread
4. Manual OAM write glitchyness (<https://forums.nesdev.org/viewtopic.php?t=10189>) thread by blargg
5. Writes to \$2003 appear to not cause OAM corruption (<https://forums.nesdev.org/viewtopic.php?p=179676#p179676>) post by lidnariq
6. \$2004 reading reliable? (<https://forums.nesdev.org/viewtopic.php?t=6424>) thread by blargg
7. \$2004 not readable on early revisions (<https://forums.nesdev.org/viewtopic.php?p=62137#p62137>) reply by jsr
8. hardware revisions and \$2004 reads (<https://forums.nesdev.org/viewtopic.php?p=150926#p150926>) reply by Great Hierophant
9. 2C07 PPU sprite evaluation notes (<https://forums.nesdev.org/viewtopic.php?t=11041>) thread by thefox
10. Problem with palette discoloration when PPU is turned off during rendering (<https://forums.nesdev.org/viewtopic.php?t=23209>) thread by N·K
11. Weird PPU writes (<https://forums.nesdev.org/viewtopic.php?p=280899#p280899>) thread by Fiskbit

Pattern tables

The **pattern table** is an area of memory connected to the PPU that defines the shapes of tiles that make up backgrounds and sprites. This data is also known as **CHR**, and the memory attached to the PPU which contains it may either be CHR-ROM or CHR-RAM. *CHR* comes from "character", as related to computer text displays where each tile might represent a single letter character.

Each tile in the pattern table is 16 bytes, made of two planes. Each bit in the first plane controls bit 0 of a pixel's color index; the corresponding bit in the second plane controls bit 1.

- If neither bit is set to 1: The pixel is background/transparent.
- If only the bit in the first plane is set to 1: The pixel's color index is 1.
- If only the bit in the second plane is set to 1: The pixel's color index is 2.
- If both bits are set to 1: The pixel's color index is 3.

This diagram depicts how a tile for $\frac{1}{2}$ (one-half fraction) is encoded, with . representing a transparent pixel.

Bit Planes	Pixel Pattern
\$0xx0=\$41 01000001	
\$0xx1=\$C2 11000010	
\$0xx2=\$44 01000100	
\$0xx3=\$48 01001000	
\$0xx4=\$10 00010000	
\$0xx5=\$20 00100000	.1....3
\$0xx6=\$40 01000000	11....3.
\$0xx7=\$80 10000000	==== .1...3..
	.1..3...
\$0xx8=\$01 00000001	==== ...3.22.
\$0xx9=\$02 00000010	..3...2
\$0xxA=\$04 00000100	.3....2.
\$0xxB=\$08 00001000	3....222
\$0xxC=\$16 00010110	
\$0xxD=\$21 00100001	
\$0xxE=\$42 01000010	
\$0xxF=\$87 10000111	

Plane 0								Plane 1							
41	01000001	00	01	00	00	00	00	00	11	00000001	01				
C2	11000010	01	01	00	00	00	00	00	11	00	00000010	02			
44	01000100	00	01	00	00	00	00	11	00	00	00000100	04			
48	01001000	00	01	00	00	11	00	00	00	00001000	08				
10	00010000	00	00	00	11	00	10	10	00	00010110	16				
20	00100000	00	00	11	00	00	00	00	10	00100001	21				
40	01000000	00	11	00	00	00	00	10	00	01000010	42				
80	10000000	11	00	00	00	00	10	10	10	10000111	87				

Encoding of a ½ tile

The pattern table is divided into two 256-tile sections: a first pattern table at \$0000-\$0FFF and a second pattern table at \$1000-\$1FFF. Occasionally, these are nicknamed the "left" and "right" pattern tables based on how emulators with a debugger display them. (See #Display convention below.)

An important aspect of a mapper's capability is how finely it allows bank switching parts of the pattern table.

Addressing

PPU addresses within the pattern tables can be decoded as follows:

```
DCBA98 76543210
-----
0HNNNN NNNPyyy
||||| |||||+--- T: Fine Y offset, the row number within a tile
||||| |||||+---- P: Bit plane (0: less significant bit; 1: more significant bit)
||+---+---+--- N: Tile number from name table
|+-----+--- H: Half of pattern table (0: "left"; 1: "right")
+-----+--- 0: Pattern table is at $0000-$1FFF
```

The value written to PPCTRL (\$2000) controls whether the background and sprites use the first pattern table (\$0000-\$0FFF) or the second pattern table (\$1000-\$1FFF). PPCTRL bit 4 applies to backgrounds, bit 3 applies to 8x8 sprites, and bit 0 of each OAM entry's tile number applies to 8x16 sprites.

For example, if rows of a tile are numbered 0 through 7, row 1 of tile \$69 in the left pattern table is stored with plane 0 in \$0691 and plane 1 in \$0699.

Display convention



Thwaite CHR ROM in a pattern table viewer

It is conventional for debugging emulators' video memory viewers to display the pattern table as two 16x16-tile grids side by side. They draw the pattern table at \$0000-\$0FFF on the left and the pattern table at \$1000-\$1FFF on the right. Each pattern table is commonly represented as a 128 by 128 pixel square, with 16 rows of 16 tiles. Usually the tiles are shown left to right, top to bottom, in Western reading order: \$00 in the top left, \$01 to the right of that, through \$0F at the top right, then \$10 through \$1F on the second row, all the way through \$FF at the bottom right. Some emulators have an option to rearrange the view for 8x16 sprites, where the first two rows are \$00, \$02, \$04, ..., \$1E, and \$01, \$03, \$05, ..., \$1F, and then each pair of rows below that shows another 16 pairs of tiles.

OAM

The OAM (Object Attribute Memory) is internal memory inside the PPU that contains a display list of up to 64 sprites, where each sprite's information occupies 4 bytes.

OAM (Sprite) Data

Byte 0 - Y position

Y position of top of sprite

Sprite data is delayed by one scanline; you must subtract 1 from the sprite's Y coordinate before writing it here. Hide a sprite by moving it down offscreen, by writing any values between #\$EF-\$FF here. Sprites are never displayed on the first line of the picture, and it is impossible to place a sprite partially off the top of the screen.

Byte 1 - Tile/index

Tile index number

For 8x8 sprites, this is the tile number of this sprite within the pattern table selected in bit 3 of PPUCTRL (\$2000).

For 8x16 sprites (bit 5 of PPUCTRL set), the PPU ignores the pattern table selection and selects a pattern table from bit 0 of this number.

```

76543210
|||||||
|||||||+- Bank ($0000 or $1000) of tiles
+++++--- Tile number of top of sprite (0 to 254; bottom half gets the next tile)

```

Thus, the pattern table memory map for 8x16 sprites looks like this:

- \$00: \$0000-\$001F
- \$01: \$1000-\$101F
- \$02: \$0020-\$003F
- \$03: \$1020-\$103F
- \$04: \$0040-\$005F
- [...]
- \$FE: \$0FE0-\$0FFF
- \$FF: \$1FE0-\$1FFF

Byte 2 - Attributes

Attributes

```

76543210
|||||||
|||||||+- Palette (4 to 7) of sprite
|||++--- Unimplemented (read 0)
||+----- Priority (0: in front of background; 1: behind background)
|+----- Flip sprite horizontally
+----- Flip sprite vertically

```

Flipping does not change the position of the sprite's bounding box, just the position of pixels within the sprite. If, for example, a sprite covers (120, 130) through (127, 137), it'll still cover the same area when flipped. In 8x16 mode, vertical flip flips each of the subtiles and also exchanges their position; the odd-numbered tile of a vertically flipped sprite is drawn on top. This behavior differs from the behavior of the unofficial 16x32 and 32x64 pixel sprite sizes on the Super NES (<https://wiki.superfamicom.org/registers#toc-8>), which will only vertically flip each square sub-region (<https://wiki.superfamicom.org/sprites>).

The three unimplemented bits of each sprite's byte 2 do not exist in the PPU and always read back as 0 on PPU revisions that allow reading PPU OAM through OAMDATA (\$2004). This can be emulated by ANDing byte 2 with \$E3 either when writing to or when reading from OAM. Bits that have decayed also read back as 0 through OAMDATA. These are side effects of the DRAM controller precharging an internal data bus with 0 to prevent writing high-impedance values to OAM DRAM cells.^[1]

Byte 3 - X position

X position of left side of sprite.

X-scroll values of \$F9-FF results in parts of the sprite to be past the right edge of the screen, thus invisible. It is not possible to have a sprite partially visible on the left edge. Instead, left-clipping through PPUMASK (\$2001) can be used to simulate this effect.

Details

DMA

Most programs write to a copy of OAM somewhere in CPU addressable RAM (often \$0200-\$02FF) and then copy it to OAM each frame using the OAMDMA (\$4014) register. Writing N to this register causes the DMA circuitry inside the 2A03/07 to fully initialize the OAM by writing OAMDATA 256 times using successive bytes from starting at address \$100*N). The CPU is suspended while the transfer is taking place.

The address range to copy from could lie outside RAM, though this is only useful for static screens with no animation.

Not counting the OAMDMA write tick, the above procedure takes 513 CPU cycles (+1 on odd CPU cycles): first one (or two) idle cycles, and then 256 pairs of alternating read/write cycles. (For comparison, an unrolled LDA/STA loop would usually take four times as long.)

Sprite 0 hits

Sprites are conventionally numbered 0 to 63. Sprite 0 is the sprite controlled by OAM addresses \$00-\$03, sprite 1 is controlled by \$04-\$07, ..., and sprite 63 is controlled by \$FC-\$FF.

While the PPU is drawing the picture, when an opaque pixel of sprite 0 overlaps an opaque pixel of the background, this is a **sprite 0 hit**. The PPU detects this condition and sets bit 6 of PPUSTATUS (\$2002) to 1 starting at this pixel, letting the CPU know how far along the PPU is in drawing the picture.

Sprite 0 hit does not happen:

- If background or sprite rendering is disabled in PPUMASK (\$2001)
- At x=0 to x=7 if the left-side clipping window is enabled (if bit 2 or bit 1 of PPUMASK is 0).
- At x=255, for an obscure reason related to the pixel pipeline.
- At any pixel where the background or sprite pixel is transparent (2-bit color index from the CHR pattern is %00).
- If sprite 0 hit has already occurred this frame. Bit 6 of PPUSTATUS (\$2002) is cleared to 0 at dot 1 of the pre-render line. This means only the first sprite 0 hit in a frame can be detected.

Sprite 0 hit happens regardless of the following:

- Sprite priority. Sprite 0 can still hit the background from behind.
- The pixel colors. Only the CHR pattern bits are relevant, not the actual rendered colors, and *any* CHR color index except %00 is considered opaque.
- The palette. The contents of the palette are irrelevant to sprite 0 hits. For example: a black (\$0F) sprite pixel can hit a black (\$0F) background as long as neither is the transparent color index %00.
- The PAL PPU blanking on the left and right edges at x=0, x=1, and x=254 (see Overscan).

Sprite overlapping

Priority between sprites is determined by their address inside OAM. So to have a sprite displayed in front of another sprite in a scanline, the sprite data that occurs first will overlap any other sprites after it. For example, when sprites at OAM \$0C and \$28 overlap, the sprite at \$0C will appear in front.

Internal operation

In addition to the primary OAM memory, the PPU contains 32 bytes (enough for 8 sprites) of secondary OAM memory that is not directly accessible by the program. During each visible scanline this secondary OAM is first cleared (set to all \$FF), and then a linear search of the entire primary OAM is carried out to find sprites that are within y range for the **next** scanline (the *sprite evaluation* phase). The OAM data for each sprite found to be within range is copied into the secondary OAM, which is then used to initialize eight internal sprite output units.

See PPU rendering for information on precise timing.

The reason sprites at lower addresses in OAM overlap sprites at higher addresses is that sprites at lower addresses also get assigned a lower address in the secondary OAM, and hence get assigned a lower-numbered sprite output unit during the loading phase. Output from lower-numbered sprite output units is wired inside the PPU to take priority over output from higher-numbered sprite output units.

Sprite 0 hit detection relies on the fact that sprite 0, when it is within y range for the next scanline, always gets assigned the first sprite output unit. The hit condition is basically *sprite 0 is in range AND the first sprite output unit is outputting a non-zero pixel AND the background drawing unit is outputting a non-zero pixel*. (Internally the PPU actually uses **two** flags: one to keep track of whether sprite 0 occurs on the *next* scanline, and another one—initialized from the first—to keep track of whether sprite 0 occurs on the *current* scanline. This is to avoid sprite evaluation, which takes place concurrently with potential sprite 0 hits, trampling on the second flag.)

Dynamic RAM decay

Because OAM is implemented with dynamic RAM instead of static RAM, the data stored in OAM memory will quickly begin to decay into random bits if it is not being refreshed. The OAM memory is refreshed once per scanline while rendering is enabled (if either the sprite or background bit is enabled via the register at \$2001), but on an NTSC PPU this refresh is prevented whenever rendering is disabled.

When rendering is turned off, or during vertical blanking between frames, the OAM memory will hold stable values for a short period before it begins to decay. It will last at least as long as an NTSC vertical blank interval (~1.3ms), but not much longer than this.^[2] Because of this, it is not normally useful to write to OAM outside of vertical blank, where rendering is expected to start refreshing its data soon after the write. Writes to \$4014 or \$2004 should usually be done in an NMI routine, or otherwise within vertical blanking.

If using an advanced technique like forced blanking to manually extend the vertical blank time, it may be necessary to do the OAM DMA last, before enabling rendering mid-frame, to avoid decay.

Because OAM decay is more or less random, and with timing that is sensitive to temperature or other environmental factors, it is not something a game could normally rely on. Most emulators do not simulate the decay, and suffer no compatibility problems as a result. Software developers targeting the NES hardware should be careful not to rely on this.

Because PAL machines have a much longer vertical blanking interval, the 2C07 (PAL PPU) forcibly refreshes OAM during scanlines 265 through 310 (starting 24 scanlines after the start of NMI^{[3][4][5]}), incrementing the OAM address once every 2 pixels (except at pixel 0). This prevents the values in DRAM from decaying in the remaining 46 scanlines before the picture starts and is long enough to allow unmodified NTSC vblank code to run correctly on PAL. As a result of this, software taking advantage of PAL's longer vblank must do OAM DMA early in vblank. In exchange, OAM decay does not occur at all on the PAL NES *provided that rendering remains enabled* - if rendering is turned off, then OAM can still decay during scanlines 311 and 0-264 (and mid-frame DMA can still be performed if necessary).

See also

- PPU sprite evaluation
- PPU sprite priority
- Sprite overflow games

References

1. "OAM" (https://github.com/emu-russia/breaks/blob/master/BreakingNESWiki_DeepL/PPU/oam.md#oam-buffer-ob) on Breaking NES Wiki. Accessed 2022-04-19.
2. Forum post: (<https://forums.nesdev.org/viewtopic.php?p=109548#p109548>) Re: Just how cranky is the PPU OAM?
3. Forum post: (<https://forums.nesdev.org/viewtopic.php?t=11041>) OAM reading on PAL NES
4. Forum post: (<https://forums.nesdev.org/viewtopic.php?t=15763>) PAL NES, sprite evaluation and \$2004 reads/writes
5. BreakingNESWiki: (https://github.com/ogamespec/breaks/blob/master/BreakingNESWiki_DeepL/PPU/pal.md) PAL circuit analysis

Nametables

A **nametable** is a 1024 byte area of memory used by the PPU to lay out backgrounds. Each byte in the nametable controls one 8x8 pixel character cell, and each nametable has 30 rows of 32 tiles each, for 960 (\$3C0) bytes; the 64 (\$40) remaining bytes are used by each nametable's attribute table. With each tile being 8x8 pixels, this makes a total of 256x240 pixels in one map, the same size as one full screen.

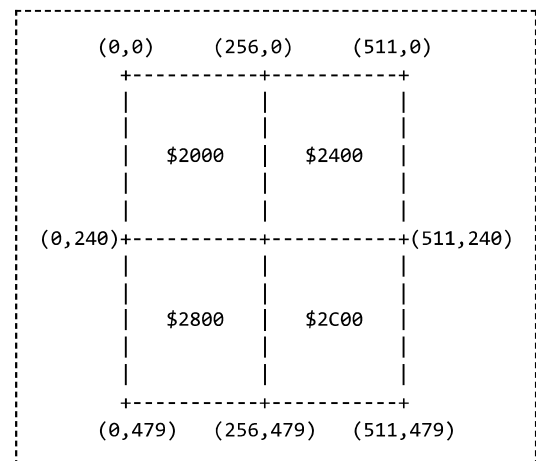
See also: PPU memory map

Mirroring

Main article: Mirroring

The NES has four logical nametables, arranged in a 2x2 pattern. Each occupies a 1 KiB chunk of PPU address space, starting at \$2000 at the top left, \$2400 at the top right, \$2800 at the bottom left, and \$2C00 at the bottom right.

But the NES system board itself has only 2 KiB of VRAM (called CIRAM, stored in a separate SRAM chip), enough for two physical nametables; hardware on the cartridge controls address bit 10 of CIRAM to map one nametable on top of another.



- Horizontal arrangement: \$2000 and \$2800 contain the first nametable, and \$2400 and \$2C00 contain the second nametable (e.g. *Super Mario Bros.*), accomplished by connecting CIRAM A10 to PPU A10
- Vertical arrangement: \$2000 and \$2400 contain the first nametable, and \$2800 and \$2C00 contain the second nametable (e.g. *Kid Icarus*), accomplished by connecting CIRAM A10 to PPU A11
- Single-screen: All nametables refer to the same memory at any given time, and the mapper directly manipulates CIRAM A10 (e.g. many Rare games using AxROM)
- Four-screen nametables: The cartridge contains additional VRAM used for all nametables (e.g. *Gauntlet*, *Rad Racer 2*)

- Other: Some advanced mappers can present arbitrary combinations of CIRAM, VRAM, or even CHR ROM in the nametable area. Such exotic setups are rarely used.

Background evaluation

Main article: PPU rendering

Conceptually, the PPU does this 33 times for each scanline:

- Fetch a nametable entry from \$2000-\$2FFF.
- Fetch the corresponding attribute table entry from \$23C0-\$2FFF and increment the current VRAM address within the same row.
- Fetch the low-order byte of an 8x1 pixel sliver of pattern table from \$0000-\$0FF7 or \$1000-\$1FF7.
- Fetch the high-order byte of this sliver from an address 8 bytes higher.
- Turn the attribute data and the pattern table data into palette indices, and combine them with data from sprite data using priority.

It also does a fetch of a 34th (nametable, attribute, pattern) tuple that is never used, but some mappers rely on this fetch for timing purposes.

See also

- PPU attribute tables

Attribute tables

An **attribute table** is a 64-byte array at the end of each nametable that controls which palette is assigned to each part of the background.

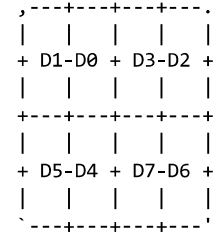
Each attribute table, starting at \$23C0, \$27C0, \$2BC0, or \$2FC0, is arranged as an 8x8 byte array:

	2xx0	2xx1	2xx2	2xx3	2xx4	2xx5	2xx6	2xx7
2xC0:	. - + - 	. - + - 	. - + - 	. - + - 	. - + - 	. - + - 	. - + - 	. - + -
2xC8:	. - + - 	. - + - 	. - + - 	. - + - 	. - + - 	. - + - 	. - + - 	. - + -
2xD0:	. - + - 	. - + - 	. - + - 	. - + - 	. - + - 	. - + - 	. - + - 	. - + -
2xD8:	. - + - 	. - + - 	. - + - 	. - + - 	. - + - 	. - + - 	. - + - 	. - + -
2xE0:	. - + - 	. - + - 	. - + - 	. - + - 	. - + - 	. - + - 	. - + - 	. - + -
2xE8:	. - + - 	. - + - 	. - + - 	. - + - 	. - + - 	. - + - 	. - + - 	. - + -

[illegible]

Each byte controls the palette of a 32×32 pixel or 4×4 tile part of the nametable and is divided into four 2-bit areas. Each area covers 16×16 pixels or 2×2 tiles, the size of a [?] block in *Super Mario Bros.* Given palette numbers topleft, topright, bottomleft, bottomright, each in the range 0 to 3, the value of the byte is

```
value = (bottomright << 6) | (bottomleft << 4) | (topright << 2) | (topleft << 0)
```



Or equivalently:

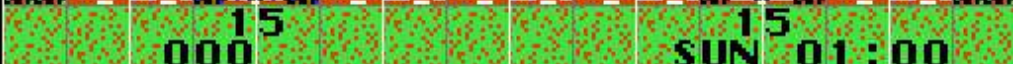
```

7654 3210
|||| ||++ Color bits 3-2 for top left quadrant of this byte
|||| ++--- Color bits 3-2 for top right quadrant of this byte
||+-+---- Color bits 3-2 for bottom left quadrant of this byte
++----- Color bits 3-2 for bottom right quadrant of this byte

```

Most games for the NES use 16×16 pixel metatiles (size of *Super Mario Bros.* ? block) or 32x32 pixel metatiles (width of **SMB** pipe) in order to align the map with the attribute areas.

Worked example

	+0	+1	+2	+3	+4	+5	+6	+7
23C0								
23C8								
23D0	Shoot down incoming missiles to defend the town!							
23D8								
23E0								
23E8								
23F0								
23F8	 15 000 15 SUN 01:00							

The background in the game *Thwaite*, with an overlaid attribute grid.

	+0	+1	+2	+3	+4	+5	+6	+7
23C0	00	00	00	00	00	00	00	00
23C8	00	00	00	00	00	00	00	00
23D0	00	00	00	00	00	00	00	00
23D8	00	00	00	00	00	00	00	00
23E0	00	00	00	00	00	00	00	00
23E8	00	00	00	00	00	00	00	00
23F0	21	23	31	23	12	31	11	23
23F8	22	22	22	22	22	22	22	22

Each 16x16 pixel color area has one of four subpalettes assigned to it, and one byte controls four color areas.

3F00 +	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Values	0f	00	10	20	0f	17	16	20	0f	17	2a	20	0f	17	12	20
Colors																
Color set	0	1	2	3												

The background palette has one backdrop color and four three-color subpalettes.

Consider the byte at \$23F2:

```
,---- Top left
3 1 - Top right
2 2 - Bottom right
`---- Bottom left
```

The byte has color set 2 at bottom right, 2 at bottom left, 1 at top right, and 3 at top left. Thus its attribute is calculated as follows:

```
value = (bottomright << 6) | (bottomleft << 4) | (topright << 2) | (topleft << 0)
       = (2      << 6) | (2      << 4) | (1      << 2) | (3      << 0)
       = $80          | $20          | $04          | $03
       = $A7
```

To find the address of an attribute byte corresponding to a nametable address, see: PPU scrolling: Tile and attribute fetching

Glitches

There are some well-known glitches when rendering attributes in NES and Famicom games.

While an attribute table specifies one of four three-color palettes for each 16x16 pixel region, the left-side clipping window in PPUMASK (\$2001) is only 8 pixels wide.

This is the reason why games that use either horizontal or vertical mirroring modes for arbitrary-direction scrolling often have color artifacts on one side of the screen (on the right side in *Super Mario Bros. 3*; on the trailing side of the scroll in *Kirby's Adventure*; and at the top and bottom in *Super C*).

The game *Alfred Chicken* hides glitches on the left and right sides by using both left clipping and hiding the right side of the screen under solid-colored sprites. To mask the entire 240-scanline height, this approach would occupy 15 entries of 64 in the sprite table in 8x16 sprite mode, or 30 entries in the 8x8 mode.

Expansion

There are two bits of memory in the attribute table that control the palette selection for each 16x16 pixel area on the screen. Because the PPU actually fetches that memory redundantly for each 8x1 pixel area as it draws the display, it is possible for a mapper to control this memory and supply different data for each read. The MMC5 mapper does this for its 8x8 extended attribute mode.

Palettes

The NES has a limited selection of color outputs. A 6-bit value in the palette memory area corresponds to one of 64 outputs. The emphasis bits of the PPUMASK register (\$2001) provide an additional color modifier.

For more information on how the colors are generated on an NTSC NES, see: NTSC video. For additional information on how the colors are generated on a PAL NES, see: PAL video.

Palette RAM

		Palette 0				Palette 1				Palette 2				Palette 3			
		\$x0	\$x1	\$x2	\$x3	\$x4	\$x5	\$x6	\$x7	\$x8	\$x9	\$xA	\$xB	\$xC	\$xD	\$xE	\$xF
Background	\$3F0x	0*	1	2	3	0	1	2	3	0	1	2	3	0	1	2	3
Sprite	\$3F1x		1	2	3		1	2	3		1	2	3		1	2	3

*** Note: Entry 0 of palette 0 is used as the backdrop color.**

Backgrounds and sprites each have 4 palettes of 4 colors, located at \$3F00-\$3F1F in VRAM. Each byte in this palette RAM contains a 6-bit color value referencing one of the PPU's 64 colors. Entry 0 of each palette is unique in that it is transparent, so its color value is normally unused. However, because of the PPU's EXT functionality, entry 0

of palette 0 has the unique behavior of being the *backdrop color*. The backdrop color is the single color shown behind both the background and sprites, wherever both layers are transparent. Artistically, the backdrop color is usually considered a fourth color of the background and is sometimes called the *universal background color*.

A single element on the screen can only use a single palette. For backgrounds, this is usually a 16x16 pixel region, but may be as small as an 8x1 pixel *sliver* with assistance from a cartridge mapper. For sprites, this is a single sprite object, which is 8x8 or 8x16 pixels, depending on the current sprite size. The palette is selected with a 2-bit value referred to as *attributes*.

Ultimately, the color of a pixel is determined by background vs sprite (which selects the set of 4 palettes), the 2 bits of attributes (which select 1 of those 4 palettes), and the 2 bits of graphics or tile *pattern* data (which select the color from that palette). These create a 5-bit index into palette RAM:

```

4bit0
-----
SAAPP
||||
|||++- Pixel value from tile pattern data
|+---- Palette number from attributes
+----- Background/Sprite select

```

Note that entry 0 of each palette is also unique in that its color value is shared between the background and sprite palettes, so writing to either one updates the same internal storage. This means that the backdrop color can be written through both \$3F00 and \$3F10. Palette RAM as a whole is also mirrored through the entire \$3F00-\$3FFF region.

Color Value Significance (Hue / Value)

As in some second-generation game consoles, values in the NES palette are based on hue and brightness:

```

5 bit 0
-----
VV HHHH
|| ||||
|| +++- Hue (phase, determines NTSC/PAL chroma)
++----- Value (voltage, determines NTSC/PAL luma)

```

Hue \$0 is light gray, \$1-\$C are blue to red to green to cyan, \$D is dark gray, and \$E-\$F are mirrors of \$1D (black).

It works this way because of the way colors are represented in a composite NTSC or PAL signal, with the phase of a color subcarrier controlling the hue. For details regarding signal generation and color decoding, see NTSC video.

The canonical code for "black" is \$0F.

The 2C03 RGB PPU used in the PlayChoice-10 and 2C05-99 in the Famicom Titler renders hue \$D as black, not dark gray. The 2C04 PPUs used in many Vs. System arcade games have completely different palettes as a copy protection measure.

Palettes

The 2C02 (NTSC) and 2C07 (PAL) PPU is used to generate an analog composite video signal. These were used in most home consoles.

The 2C03, 2C04, and 2C05, on the other hand, all output analog red, green, blue, and sync (RGS) signals. The sync signal contains horizontal and vertical sync pulses in the same format as an all-black composite signal. Each of the three video channels uses a 3-bit DAC driven by a look-up table in a 64x9-bit ROM inside the PPU. The look-up tables (one digit for each of red, green, and blue, in order) are given below.

RGB PPUs were used mostly in arcade machines (e.g. Vs. System, Playchoice 10), as well as the Sharp Famicom Titler.

2C02

The RF Famicom, AV Famicom, NES (both front- and top-loading), and the North American version of the Sharp Nintendo TV use the 2C02 PPU. Unlike some other consoles' video circuits, the 2C02 does not generate RGB video and then encode that to composite. Instead it generates NTSC video directly in the composite domain, decoded by the television receiver into RGB to drive its picture tube.



Do not use color \$0D. It results in a "blacker than black" signal that may cause problems for some TVs.

Further details on \$0D and its effects can be found [here](#).

Most emulators can use a predefined palette, such as one commonly stored in common .pal format, in which each triplet represents the 8bpc sRGB color that results from decoding a large flat area with a given palette value.

The palette seen on real hardware connected to a real television set has at least four sources of variation:

- variation in impedance mismatch between console and television set; this variance leads to different signal reflections with different differential phase distortion, which manifests itself in brighter colors losing saturation and shifting in hue.
- variation in user-adjustable settings of the television set (in particular, hue and saturation);
- variation in how television sets decode composite video into their native RGB display colorspace;
- variation in the native RGB colorspaces of television sets (red/green/blue phosphor chromaticities, color temperature).

The combined effect is that no single composite palette can match the intended display of every possible game title. Nintendo never specified a reference monitor to licensed developers, either. Emulators reproducing a palette or a signal decoding chain must decide on parameters for differential phase distortion, user-adjustable hue and saturation, composite decoding and assumed monitor colorimetry. For composite decoding and assumed monitor colorimetry, the coefficients from published standards such as SMPTE 170M can serve as reasonable defaults.

See this page for more details on a general algorithm to decode a PPU composite signal to color RGB.

The following palette was generated using Pally v0.22.1 (<https://github.com/Gumball2415/palgen-persune>) with the following arguments:

```
pally.py --skip-plot -cld -phd 4 -e -o docs/NESDev/2C02G_wiki.pal
```

Download: [File:2C02G wiki.pal](#)

\$00	\$01	\$02	\$03	\$04	\$05	\$06	\$07	\$08	\$09	\$0A	\$0B	\$0C	\$0D	\$0E	\$0F
\$10	\$11	\$12	\$13	\$14	\$15	\$16	\$17	\$18	\$19	\$1A	\$1B	\$1C	\$1D	\$1E	\$1F
\$20	\$21	\$22	\$23	\$24	\$25	\$26	\$27	\$28	\$29	\$2A	\$2B	\$2C	\$2D	\$2E	\$2F
\$30	\$31	\$32	\$33	\$34	\$35	\$36	\$37	\$38	\$39	\$3A	\$3B	\$3C	\$3D	\$3E	\$3F

Other tools for generating a palette include one by Bisqwit (<http://bisqwit.iki.fi/utils/nespalette.php>) and one by Drag (<http://drag.wootest.net/misc/palgen.html>). These simulate generating a large area of one flat color and then decoding that with the adjustment knobs set to various settings.

2C07

The PAL PPU (2C07) generates a composite PAL video signal, which has a -15 degree hue shift relative to the 2C02 due to a different colorburst reference phase generated by the PPU (\$x7 rather than \$x8), in addition to the PAL colorburst phase being defined as $-U \pm 45$ degrees.

Like NTSC, the PAL composite signal is affected by differential phase distortion to the same degree, if not slightly worse. Unlike NTSC, PAL composite has a mechanism to correct hue errors by shifting the phase of the colorburst reference on every line; thus the colors on the 2C07 will remain consistent on most PAL sets at the cost of slight further saturation loss.

The following palette was generated using Pally v0.22.1 (<https://github.com/Gumball2415/palgen-persune>) with the following arguments:

```
python3 pally.py --skip-plot -cld -ppu "2C07" -phd 4 --delay-line-filter -e -o docs/NESDev/2C07_wiki.pal
```

Download: [File:2C07 wiki.pal](#)

\$00	\$01	\$02	\$03	\$04	\$05	\$06	\$07	\$08	\$09	\$0A	\$0B	\$0C	\$0D	\$0E	\$0F
\$10	\$11	\$12	\$13	\$14	\$15	\$16	\$17	\$18	\$19	\$1A	\$1B	\$1C	\$1D	\$1E	\$1F
\$20	\$21	\$22	\$23	\$24	\$25	\$26	\$27	\$28	\$29	\$2A	\$2B	\$2C	\$2D	\$2E	\$2F
\$30	\$31	\$32	\$33	\$34	\$35	\$36	\$37	\$38	\$39	\$3A	\$3B	\$3C	\$3D	\$3E	\$3F

2C03 and 2C05

This palette is intentionally similar to the NES's standard palette, but notably is missing the greys in entries \$2D and \$3D. The 2C03 is used in *Vs. Duck Hunt*, *Vs. Tennis*, all PlayChoice games, and the Sharp C1 (Famicom TV). The 2C05 is used in some later Vs. games as a copy protection measure. A variant of the 2C05 without copy protection measures is present in the Sharp Famicom Titler, albeit with adjustments to the output (see below). Both have historically been used in RGB mods for the NES, as a circuit implementing $A0' = A0 \text{ xor } (A1 \text{ nor } A2)$ can swap PPUCTRL and PPUMASK to make a 2C05 behave as a 2C03.

The formula for mapping the DAC integer channel value to 8-bit per channel color is $C = 255 * DAC / 7$.

Download: File:2C03 wiki.pal

	\$x0	\$x1	\$x2	\$x3	\$x4	\$x5	\$x6	\$x7	\$x8	\$x9	\$xA	\$xB	\$xC	\$xD	\$xE	\$xF
\$0x	333	014	006	326	403	503	510	420	320	120	031	040	022	000	000	000
\$1x	555	036	027	407	507	704	700	630	430	140	040	053	044	000	000	000
\$2x	777	357	447	637	707	737	740	750	660	360	070	276	077	000	000	000
\$3x	777	567	657	757	747	755	764	772	773	572	473	276	467	000	000	000

Note that some of the colors are duplicates: \$0B and \$1A = 040, \$2B and \$3B = 276.

Monochrome works the same as the 2C02 (consistent across **all** RGB PPUs), but unlike the 2C02, emphasis on the RGB chips works differently; rather than "darkening" the specific color chosen, it sets the corresponding channel to full brightness instead.

2C05-99

The Sharp Famicom Titler (AN-510) contains a RC2C05-99 PPU, whose RGB output is fed into a X0858CE transcoder chip. This chip handles the conversion from RGBS into Composite, and S-Video. The additional components outside the chip reduce the saturation of the output palette by half with the use of a voltage divider^[1], specifically both the R-Y and B-Y color difference channels on pin 11 and pin 12 ($U = U * 0.5$; $V = V * 0.5$). This was likely done due to the highly saturated colors of RGB video, especially the 2C05-99's raw RGBS output, which may cause excessive color bleed on composite video.

Otherwise, the raw internal palette of the 2C05-99 PPU itself is believed to be identical to that of the 2C03.

\$00	\$01	\$02	\$03	\$04	\$05	\$06	\$07	\$08	\$09	\$0A	\$0B	\$0C	\$0D	\$0E	\$0F
\$10	\$11	\$12	\$13	\$14	\$15	\$16	\$17	\$18	\$19	\$1A	\$1B	\$1C	\$1D	\$1E	\$1F
\$20	\$21	\$22	\$23	\$24	\$25	\$26	\$27	\$28	\$29	\$2A	\$2B	\$2C	\$2D	\$2E	\$2F
\$30	\$31	\$32	\$33	\$34	\$35	\$36	\$37	\$38	\$39	\$3A	\$3B	\$3C	\$3D	\$3E	\$3F

2C04

All four 2C04 PPUs contain the same master palette, but in different permutations. It's almost a superset of the 2C03/5 palette, adding four greys, six other colors, and making the bright yellow more pure.

Much like the 2C03 and 2C02, using the greyscale bit in PPUMASK will remap the palette by index, not by color. This means that with the scrambled palettes, each row will remap to the colors in the \$0X column for that 2C04 version.

Visualization tool: RGB PPU Palette Converter (<https://www.nesdev.org/rgbppu/>)

No version of the 2C04 was ever made with the below ordering, but it shows the similarity to the 2C03:

333	014	006	326	403	503	510	420	320	120	031	040	022	111	003	020
555	036	027	407	507	704	700	630	430	140	040	053	044	222	200	310
777	357	447	637	707	737	740	750	660	360	070	276	077	444	000	000
777	567	657	757	747	755	764	770	773	572	473	276	467	666	653	760

The PPUMASK monochrome bit has the same implementation as on the 2C02, and so it has an unintuitive effect on the 2C04 PPUs; rather than forcing colors to grayscale, it instead forces them to the first column.

RP2C04-0001

MAME's source claims that *Baseball*, *Freedom Force*, *Gradius*, *Hogan's Alley*, *Mach Rider*, *Pinball*, and *Platoon* require this palette.

755,637,700,447,044,120,222,704,777,333,750,503,403,660,320,777
357,653,310,360,467,657,764,027,760,276,000,200,666,444,707,014
003,567,757,070,077,022,053,507,000,420,747,510,407,006,740,000
000,140,555,031,572,326,770,630,020,036,040,111,773,737,430,473

Palette colors

[Expand]

RP2C04-0002

MAME's source claims that *Castlevania*, *Mach Rider (Endurance Course)*, *Raid on Bungeling Bay*, *Slalom*, *Soccer*, *Stroke & Match Golf* (both versions), and *Wrecking Crew* require this palette.

000,750,430,572,473,737,044,567,700,407,773,747,777,637,467,040
020,357,510,666,053,360,200,447,222,707,003,276,657,320,000,326
403,764,740,757,036,310,555,006,507,760,333,120,027,000,660,777
653,111,070,630,022,014,704,140,000,077,420,770,755,503,031,444

Palette colors

[Expand]

RP2C04-0003

MAME's source claims that *Balloon Fight*, *Dr. Mario*, *Excitebike (US)*, *Goonies*, and *Soccer* require this palette.

507,737,473,555,040,777,567,120,014,000,764,320,704,666,653,467
447,044,503,027,140,430,630,053,333,326,000,006,700,510,747,755
637,020,003,770,111,750,740,777,360,403,357,707,036,444,000,310
077,200,572,757,420,070,660,222,031,000,657,773,407,276,760,022

Palette colors

[Expand]

RP2C04-0004

MAME's source claims that *Clu Clu Land*, *Excitebike* (Japan), *Ice Climber* (both versions), and *Super Mario Bros.* require this palette.

```
430,326,044,660,000,755,014,630,555,310,070,003,764,770,040,572
737,200,027,747,000,222,510,740,653,053,447,140,403,000,473,357
503,031,420,006,407,507,333,704,022,666,036,020,111,773,444,707
757,777,320,700,760,276,777,467,000,750,637,567,360,657,077,120
```

Palette colors

[\[Expand\]](#)

Games compatible with multiple different PPUs

Some games don't require that arcade owners have the correct physical PPU.

At the very least, the following games use some of the DIP switches to support multiple different PPUs:

- Atari RBI Baseball
- Battle City
- Star Luster
- Super Sky Kid
- Super Xevious
- Tetris (Tengen)
- TKO Boxing

LUT approach

Emulator authors may implement the 2C04 variants as a LUT indexing the "ordered" palette. This has the added advantage of being able to use preexisting .pal files if the end user wishes to do so.

Repeating colors such as 000 and 777 may index into the same entry of the "ordered" palette, as this is functionally identical.

```
const unsigned char PaletteLUT_2C04_0001 [64] ={
    0x35,0x23,0x16,0x22,0x1C,0x09,0x1D,0x15,0x20,0x00,0x27,0x05,0x04,0x28,0x08,0x20,
    0x21,0x3E,0x1F,0x29,0x3C,0x32,0x36,0x12,0x3F,0x2B,0x2E,0x1E,0x3D,0x2D,0x24,0x01,
    0x0E,0x31,0x33,0x2A,0x2C,0x0C,0x1B,0x14,0x2E,0x07,0x34,0x06,0x13,0x02,0x26,0x2E,
    0x2E,0x19,0x10,0x0A,0x39,0x03,0x37,0x17,0x0F,0x11,0x0B,0x0D,0x38,0x25,0x18,0x3A
};

const unsigned char PaletteLUT_2C04_0002 [64] ={
    0x2E,0x27,0x18,0x39,0x3A,0x25,0x1C,0x31,0x16,0x13,0x38,0x34,0x20,0x23,0x3C,0x0B,
    0x0F,0x21,0x06,0x3D,0x1B,0x29,0x1E,0x22,0x1D,0x24,0x0E,0x2B,0x32,0x08,0x2E,0x03,
    0x04,0x36,0x26,0x33,0x11,0x1F,0x10,0x02,0x14,0x3F,0x00,0x09,0x12,0x2E,0x28,0x20,
    0x3E,0x0D,0x2A,0x17,0x0C,0x01,0x15,0x19,0x2E,0x2C,0x07,0x37,0x35,0x05,0x0A,0x2D
};

const unsigned char PaletteLUT_2C04_0003 [64] ={
    0x14,0x25,0x3A,0x10,0x0B,0x20,0x31,0x09,0x01,0x2E,0x36,0x08,0x15,0x3D,0x3E,0x3C,
    0x22,0x1C,0x05,0x12,0x19,0x18,0x17,0x1B,0x00,0x03,0x2E,0x02,0x16,0x06,0x34,0x35,
    0x23,0x0F,0x0E,0x37,0x0D,0x27,0x26,0x20,0x29,0x04,0x21,0x24,0x11,0x2D,0x2E,0x1F,
    0x2C,0x1E,0x39,0x33,0x07,0x2A,0x28,0x1D,0x0A,0x2E,0x32,0x38,0x13,0x2B,0x3F,0x0C
};

const unsigned char PaletteLUT_2C04_0004 [64] ={
    0x18,0x03,0x1C,0x28,0x2E,0x35,0x01,0x17,0x10,0x1F,0x2A,0x0E,0x36,0x37,0x0B,0x39,
```

```
0x25,0x1E,0x12,0x34,0x2E,0x1D,0x06,0x26,0x3E,0x1B,0x22,0x19,0x04,0x2E,0x3A,0x21,
0x05,0x0A,0x07,0x02,0x13,0x14,0x00,0x15,0x0C,0x3D,0x11,0x0F,0x0D,0x38,0x2D,0x24,
0x33,0x20,0x08,0x16,0x3F,0x2B,0x20,0x3C,0x2E,0x27,0x23,0x31,0x29,0x32,0x2C,0x09
};
```

Backdrop color

The backdrop color (\$3F00) is shown wherever backgrounds and sprites are both transparent, as well as in the border region on NTSC and RGB PPUs. When only one of background or sprite rendering is disabled, the disabled component is treated as transparent. Disabling components in the left column via PPUMASK also treats them as transparent there.

Backdrop override

When both background and sprite rendering are disabled, this is called forced blank. During forced blank, the PPU normally draws the backdrop color. However, if the current VRAM address in *v* points into palette RAM (\$3F00-\$3FFF), then the color at that address will be drawn, instead, overriding the backdrop color. This can allow the CPU to deliberately select colors to draw to the screen, including in the border region, for effects such as color bars or to draw more colors on the screen than is possible during rendering. It can also be used to show the normally-unused transparent colors in \$3Fx4/\$3Fx8/\$3FxC.

More commonly, though, backdrop override results in a glitch where updating palettes outside of vblank causes the palette to be briefly drawn on the screen. Because of this, palette updates should be timed to occur during vblank. Backdrop override can also occur naturally based on scroll position if *v* points into palette RAM when rendering disables, usually resulting in colors flashing in the 2 scanlines of border at the bottom of the screen. Fortunately, this border flashing is normally hidden by overscan.

Backdrop override is not a deliberate PPU feature, but rather a side effect of how palette RAM is addressed. Palette RAM has just one address input, which is normally the address produced by the rendering hardware. However, when palettes are written by the CPU, it instead needs to use the CPU's target address. The PPU draws whatever color is output from palette RAM, so changing the address to allow CPU access also changes the color drawn to the screen. This is similar to color RAM dots on some Sega consoles, but occurs continuously rather than just during the CPU access.

Color names

When programmers and artists are communicating, it's often useful to have human-readable names for colors. Many graphic designers who have done web or game work will be familiar with HTML color names.

Luma

- \$0F: Black
- \$00: Dark gray
- \$10: Light gray or silver
- \$20: White
- \$01-\$0C: Dark colors, medium mixed with black
- \$11-\$1C: Medium colors, similar brightness to dark gray
- \$21-\$2C: Light colors, similar brightness to light gray
- \$31-\$3C: Pale colors, light mixed with white

Chroma

Names for hues:

- \$x0: Gray
- \$x1: Azure
- \$x2: Blue
- \$x3: Violet
- \$x4: Magenta
- \$x5: Rose
- \$x6: Red or maroon
- \$x7: Orange
- \$x8: Yellow or olive
- \$x9: Chartreuse
- \$xA: Green
- \$xB: Spring
- \$xC: Cyan

RGBI

These NES colors approximate colors in 16-color RGBI palettes, such as the CGA, EGA, or classic Windows palette, though the NES doesn't really have particularly good approximations:

- \$0F: 0/Black
- \$02: 1/Navy
- \$1A: 2/Green
- \$1C: 3/Teal
- \$06: 4/Maroon
- \$14: 5/Purple
- \$18: 6/Olive (\$17 for the brown in CGA/EGA in RGB)
- \$10: 7/Silver
- \$00: 8/Gray
- \$12: 9/Blue
- \$2A: 10/Lime
- \$2C: 11/Aqua/Cyan
- \$16: 12/Red
- \$24: 13/Fuchsia/Magenta
- \$28: 14/Yellow
- \$30: 15/White

See also

- NTSC video - details of the NTSC signal generation and how it produces the palette
- PAL video
- Palettes (gallery) - a few different visualizations of PPU palettes.
- Another palette test (<https://forums.nesdev.org/viewtopic.php?t=13264>) - Simple test ROM to display the palette.
- Full palette demo - Demo that displays the entire palette with all emphasis settings at once.
- RGB PPU Palette Converter (<https://www.nesdev.org/rgbppu/>) - RGB PPU palette conversion and visualization tool, written by WhoaMan (<https://forums.nesdev.org/viewtopic.php?p=104217>).

References

- Re: Various questions about the color palette and emulators (<https://forums.nesdev.org/viewtopic.php?p=98955#p98955>) - 2012 collection of 2C03, 2C04, 2C05 palettes.

Memory map

PPU memory map

The PPU addresses a 14-bit (16kB) address space, \$0000-\$3FFF, completely separate from the CPU's address bus. It is either directly accessed by the PPU itself, or via the CPU with memory mapped registers at \$2006 and \$2007.

Address range	Size	Description	Mapped by
\$0000-\$0FFF	\$1000	Pattern table 0	Cartridge
\$1000-\$1FFF	\$1000	Pattern table 1	Cartridge
\$2000-\$23BF	\$03c0	Nametable 0	Cartridge
\$23C0-\$23FF	\$0040	Attribute table 0	Cartridge
\$2400-\$27BF	\$03c0	Nametable 1	Cartridge
\$27C0-\$27FF	\$0040	Attribute table 1	Cartridge
\$2800-\$2BBF	\$03c0	Nametable 2	Cartridge
\$2BC0-\$2BFF	\$0040	Attribute table 2	Cartridge
\$2C00-\$2FBF	\$03c0	Nametable 3	Cartridge
\$2FC0-\$2FFF	\$0040	Attribute table 3	Cartridge
\$3000-\$3EFF	\$0F00	Unused	Cartridge
\$3F00-\$3F1F	\$0020	Palette RAM indexes	Internal to PPU
\$3F20-\$3FFF	\$00E0	Mirrors of \$3F00-\$3F1F	Internal to PPU

Hardware mapping

The NES has 2kB of RAM dedicated to the PPU, usually mapped to the nametable address space from \$2000-\$2FFF, but this can be rerouted through custom cartridge wiring. The mappings above are the addresses from which the PPU uses to fetch data during rendering. The actual devices that the PPU fetches pattern, name table and attribute table data from is configured by the cartridge.

- \$0000-1FFF is normally mapped by the cartridge to a CHR-ROM or CHR-RAM, often with a bank switching mechanism.
- \$2000-2FFF is normally mapped to the 2kB NES internal VRAM, providing 2 nametables with a mirroring configuration controlled by the cartridge, but it can be partly or fully remapped to ROM or RAM on the cartridge, allowing up to 4 simultaneous nametables.
- \$3000-3EFF is usually a mirror of the 2kB region from \$2000-2EFF. The PPU does not render from this address range, so this space has negligible utility.
- \$3F00-3FFF is not configurable, always mapped to the internal palette control.

OAM

In addition, the PPU internally contains 256 bytes of memory known as Object Attribute Memory which determines how sprites are rendered. The CPU can manipulate this memory through memory mapped registers at OAMADDR (\$2003), OAMDATA (\$2004), and OAMDMA (\$4014). OAM can be viewed as an array with 64 entries. Each entry has 4 bytes: the sprite Y coordinate, the sprite tile number, the sprite attribute, and the sprite X coordinate.

Address Low Nibble	Description
\$0, \$4, \$8, \$C	Sprite Y coordinate
\$1, \$5, \$9, \$D	Sprite tile #
\$2, \$6, \$A, \$E	Sprite attribute
\$3, \$7, \$B, \$F	Sprite X coordinate

1. Sharp CZ-880 service manual schematic, which uses the same X0858CE transcoder chip and voltage divider resistors on the color-difference I/O pins (https://eaw.app/Downloads/Manuals/Sharp/CZ-880_Service_Manual.pdf#page=52)

Retrieved from "https://www.nesdev.org/w/index.php?title=PPU_programmer_reference&oldid=9986"

This page was last edited on 29 March 2010, at 05:23.